

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

PROGRAMACIÓN 3

14va. práctica (tipo b)

(Primer Semestre 2026)

Indicaciones Generales:

- **Duración 1h 50 minutos**
- Se les recuerda que, de acuerdo con el reglamento disciplinario de nuestra institución, constituye una falta grave copiar del trabajo realizado por otro estudiante o cometer plagio para el desarrollo de esta práctica.
- Para el desarrollo de toda la práctica debe utilizar el sistema operativo Windows, Java 25 y .NET 10.
- Para el desarrollo de las preguntas debe usar IntelliJ y Visual Studio.
- Para el desarrollo de la práctica puede usar sentencias SQL o Procedimientos Almacenados.
- Está permitido el uso de Internet (únicamente para consultar páginas oficiales de Oracle, Microsoft, Amazon Web Services). No obstante, está prohibida toda forma de comunicación con otros estudiantes o terceros.
- PUEDE UTILIZAR MATERIAL DE CONSULTA. Antes de comenzar el laboratorio, descargue todos los proyectos, apuntes, diapositivas que utilizará.
- Se considerará en la calificación el uso de buenas prácticas de programación (aquellas vistas en clase).
- Al finalizar el laboratorio debe entregar un archivo llamado credenciales.txt con el nombre del servidor, base de datos, usuario y contraseña de su base de datos.

Puntaje total: 20 puntos

Parte Práctica

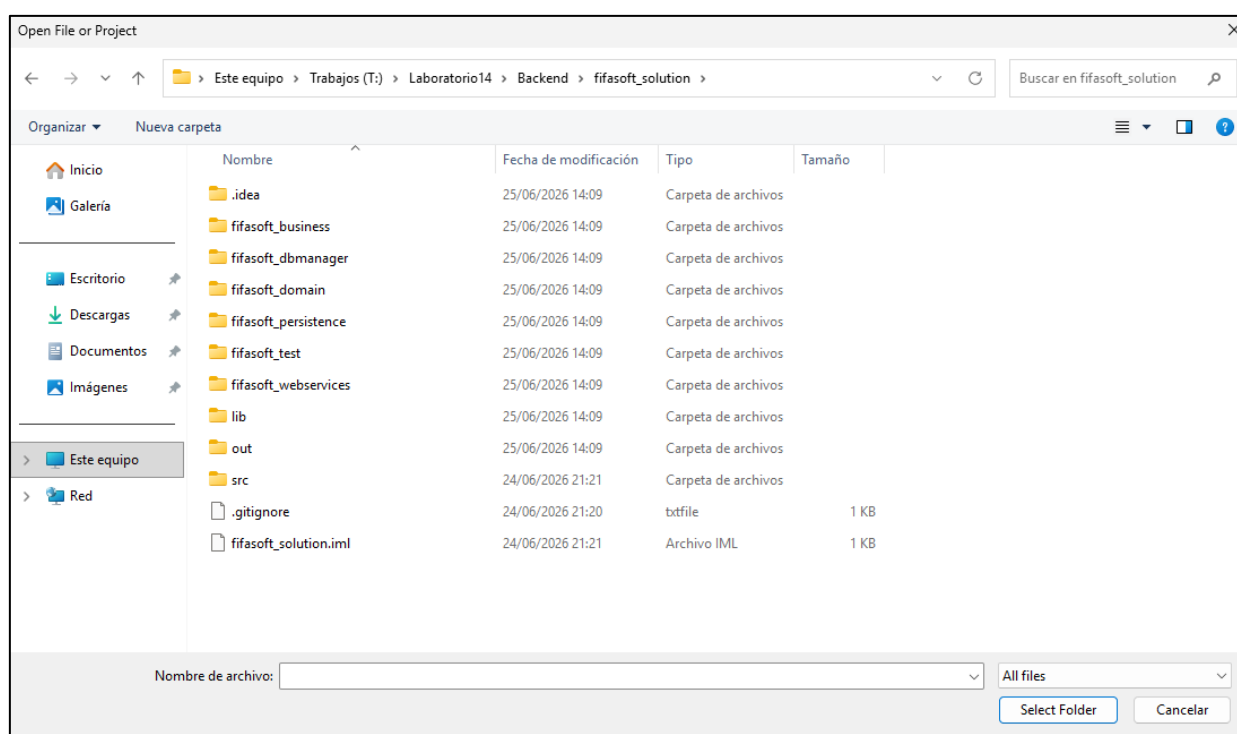
Pregunta 1 (20 puntos) – Servicios REST y Blazor

Descargar el archivo **Lab14-Prog3-2026-1.zip**. En este encontrará:

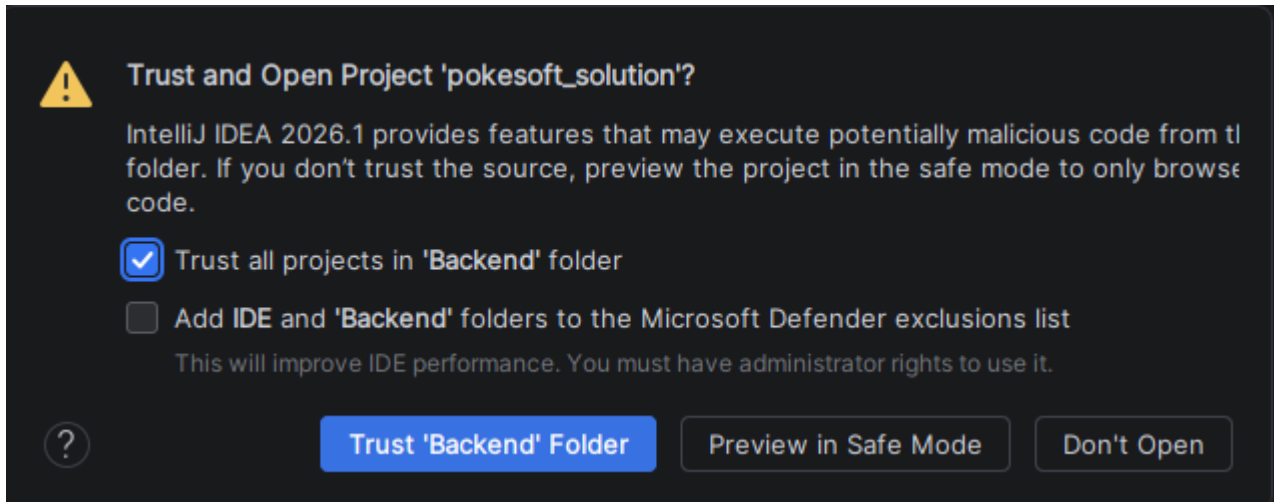
- El proyecto base en JAVA (**Backend**).
- El proyecto base en C# (**Frontend**).
- La carpeta del Glassfish (**glassfish8**). Esta carpeta **glassfish** ya tiene configurada la variable **AS_JAVA** para trabajar con la versión de **JDK 25** instalada en las máquinas de los laboratorios.
- El script SQL (**script_SQL.sql**).

Configuración:

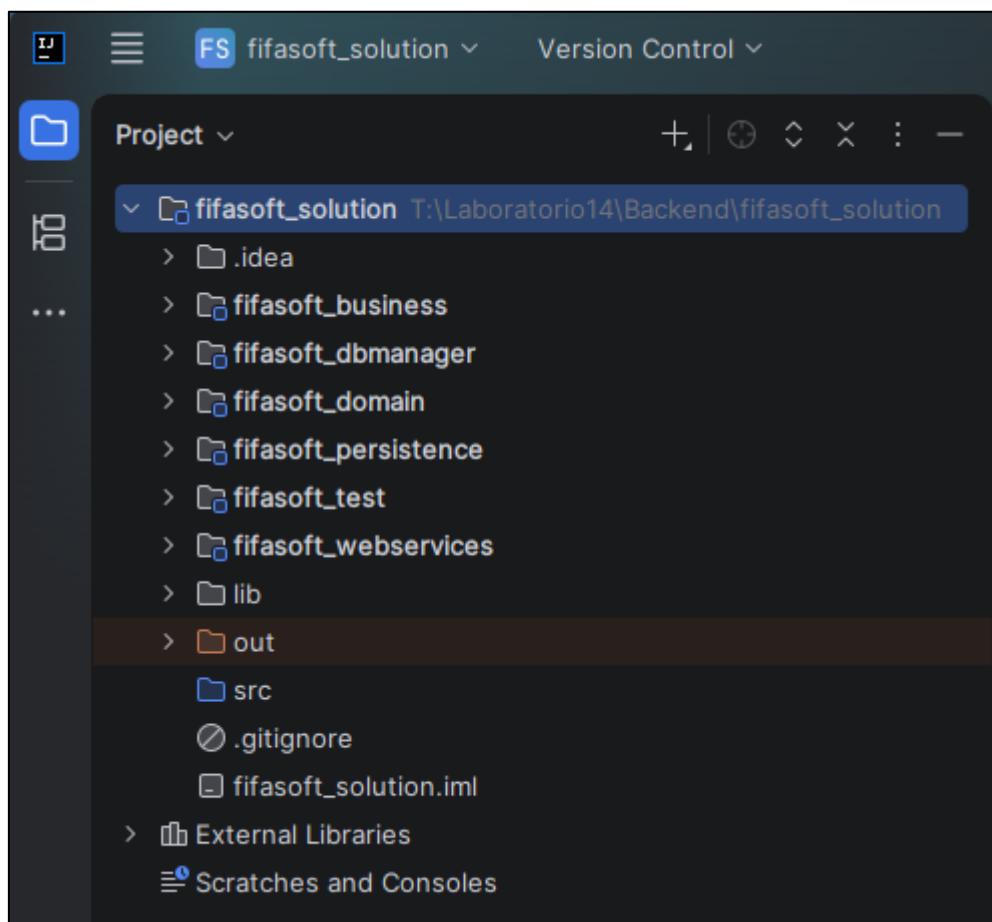
- Descomprima todos los archivos del .zip en la unidad T.
- Ahora, abra el IntelliJ IDEA 2026.
- Haga clic en "Open".
- Ubique la ruta "**T:\Laboratorio14\Backend\fifasoft_solution**" y de clic en "**Select Folder**".



- Desactive la casilla "Add IDE and 'Backend' folders to the Microsoft Defender exclusions list".
- Active la casilla "Trust all projects in 'Backend' folder".
- Clic en "Trust 'Backend' Folder".

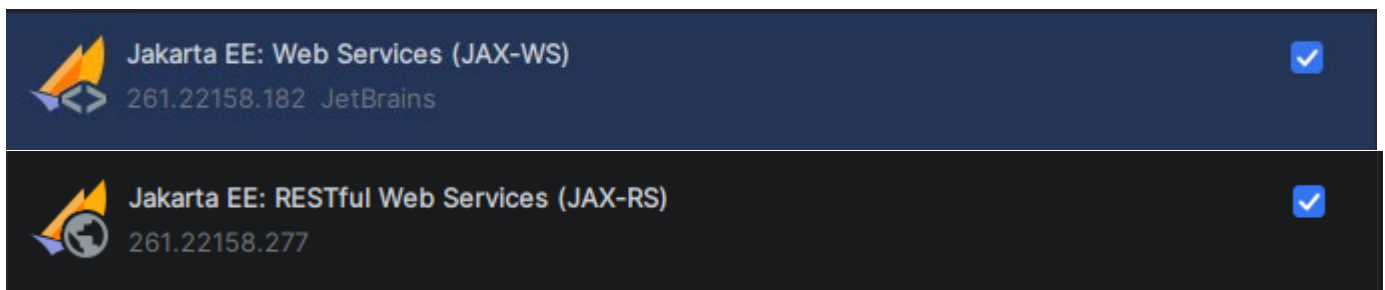


- Debería visualizar algo similar a lo siguiente:

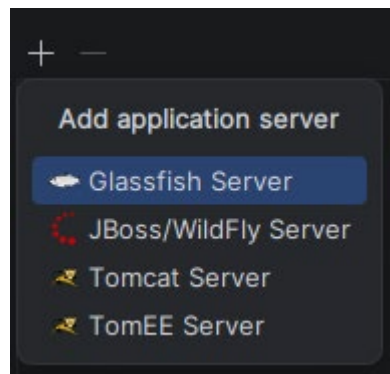


- Ingrese a "Settings" -> "Plugins" ->
 - Instale el plugin de "GlassFish".
 - Instale el plugin de "Jakarta EE: Web Services (JAX-WS)".
 - Instale el plugin de "Jakarta EE: RESTful Web Services (JAX-RS)".
 - Aplique y acepte.

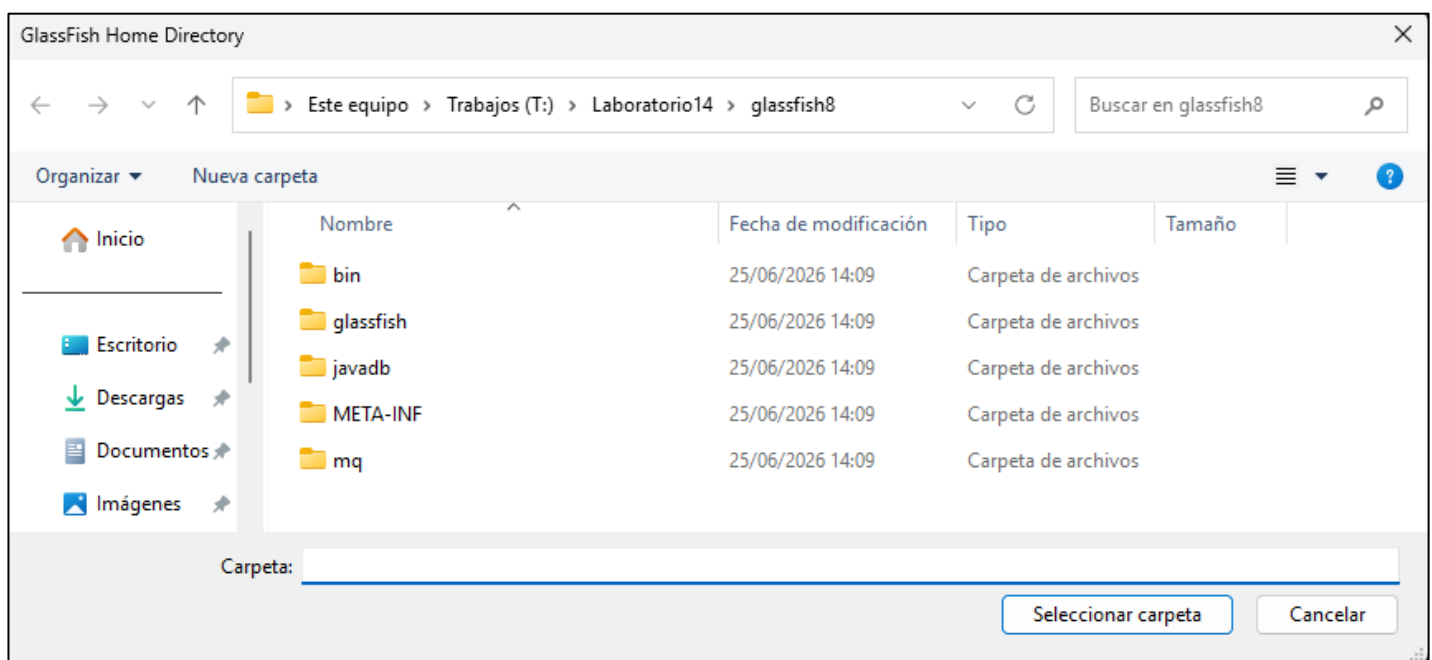




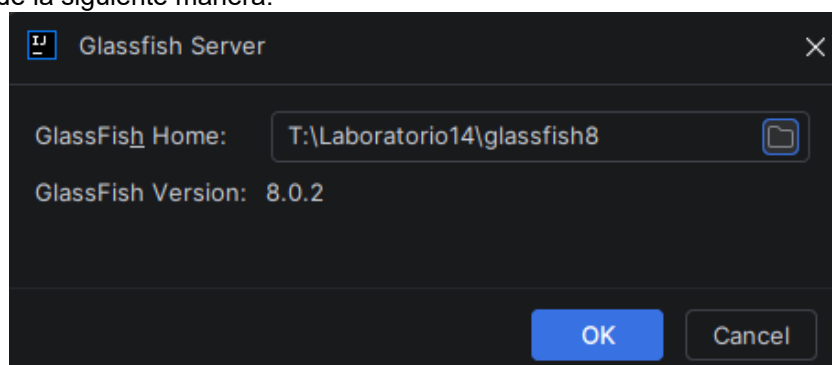
- Ingrese a “**Settings**” -> “**Build, Execution and Deployment**” -> “**Application Server**” y clic en +.
- Elegimos “**Glassfish Server**”.



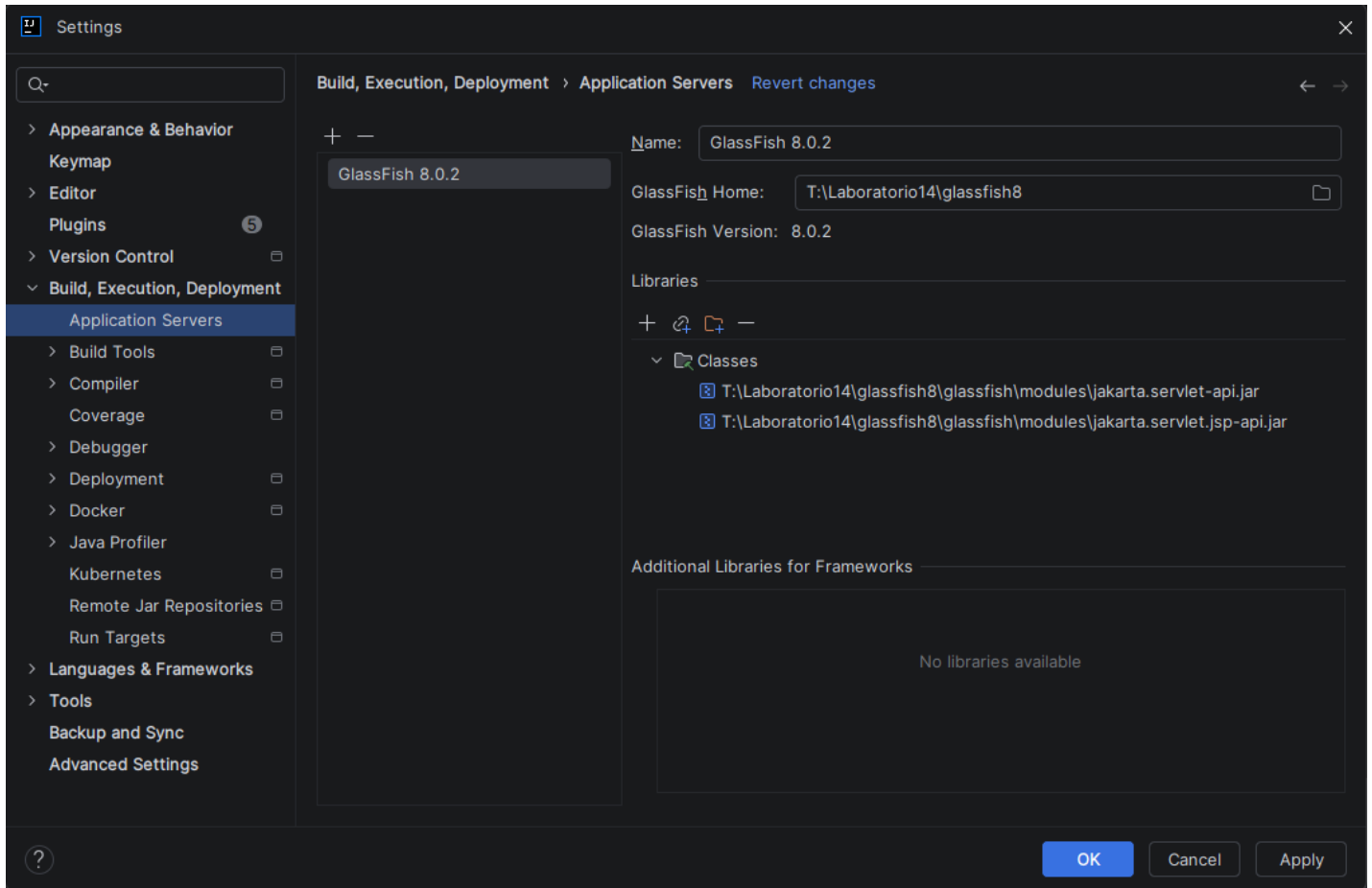
- Ubicamos la carpeta donde se encuentra el Glassfish:
 - **T:\Laboratorio14\glassfish8**
 - Damos clic en “**Seleccionar carpeta**”



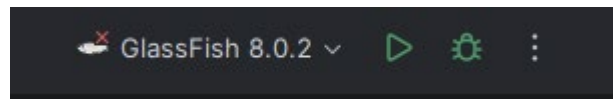
- Nos aparecerá de la siguiente manera:



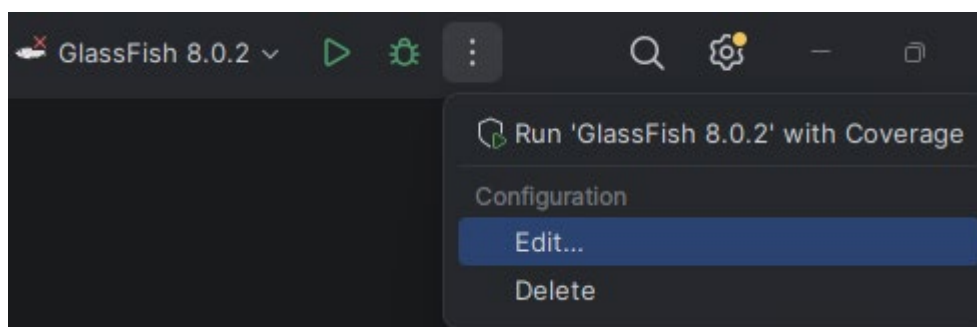
- Damos clic en OK.



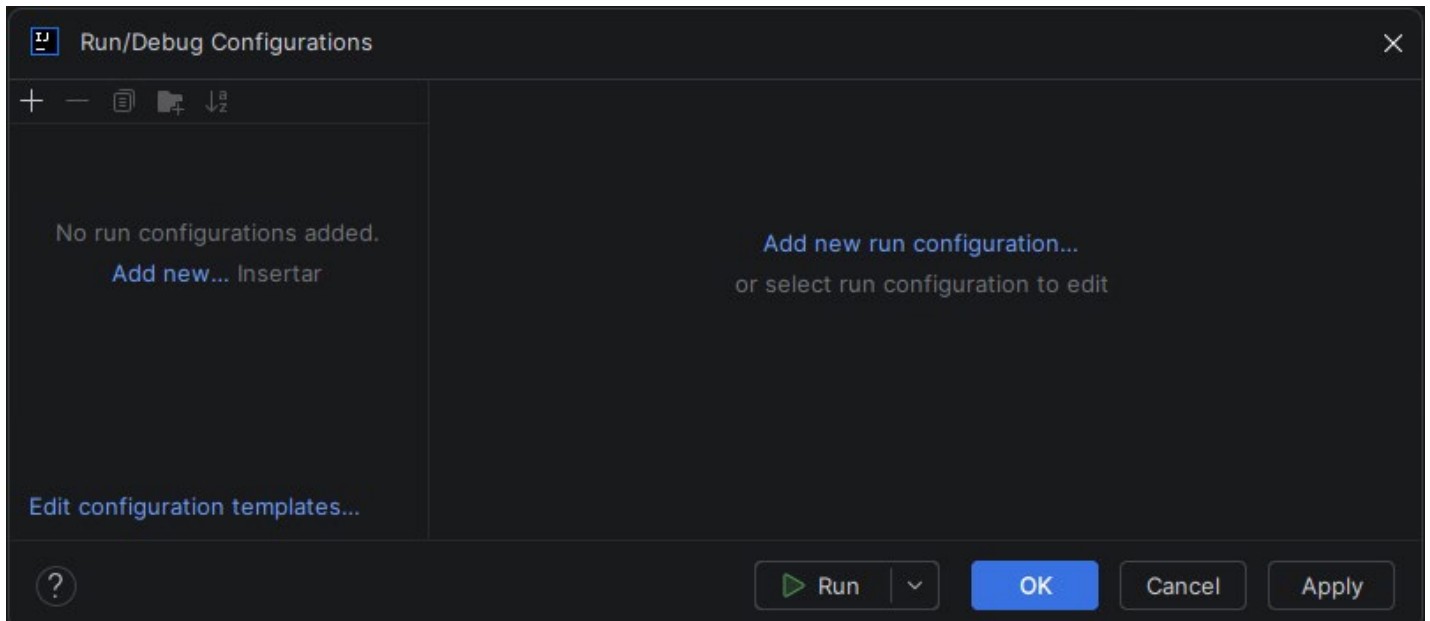
- Ahora damos clic en el botón superior de los 3 puntos.



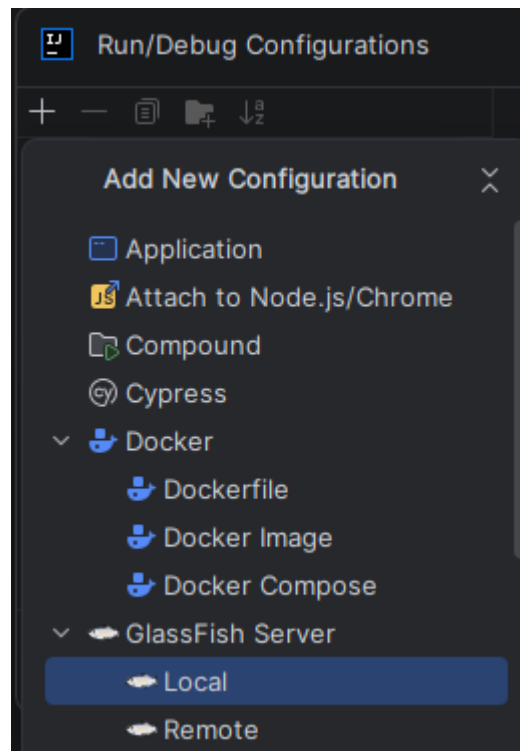
- Damos clic en “**Edit...**”



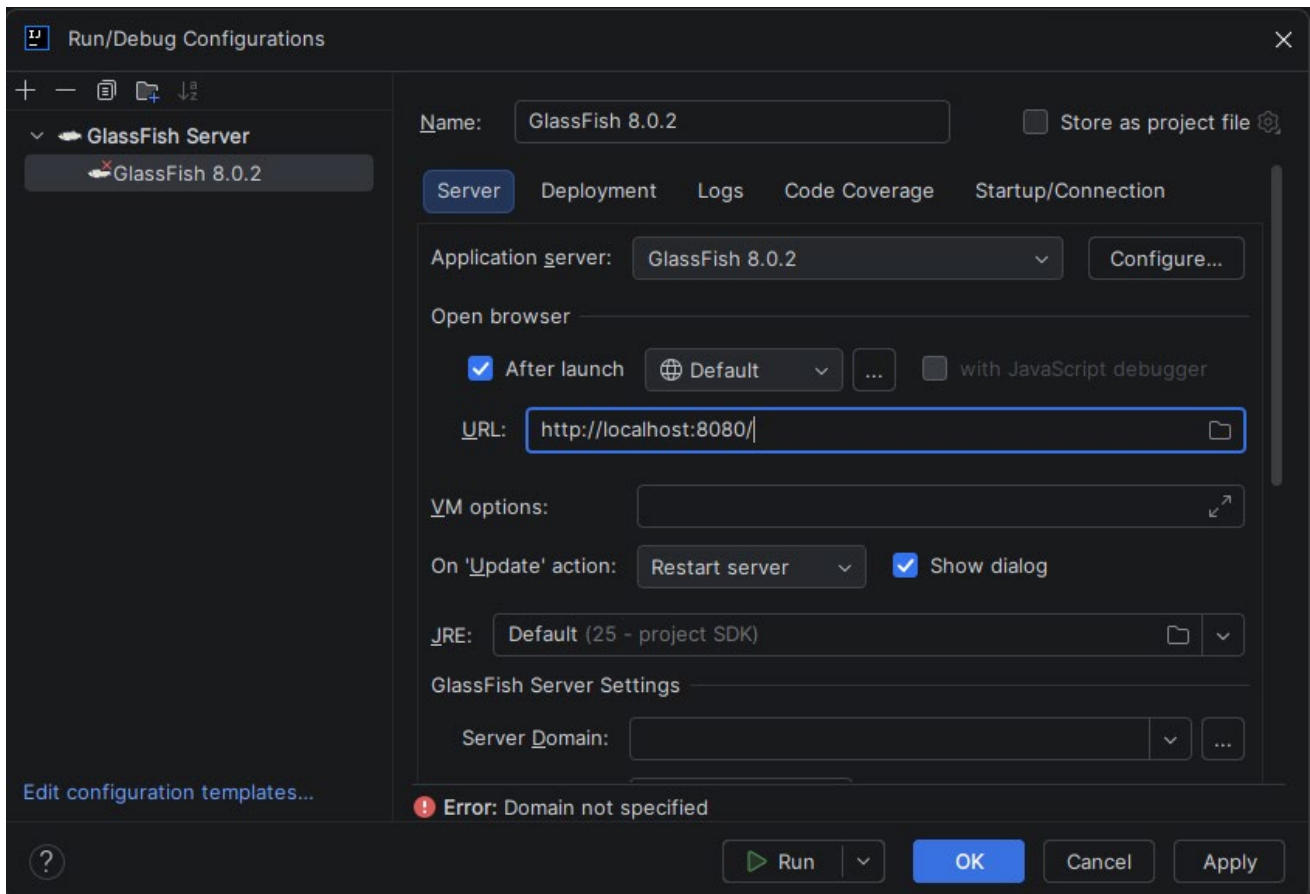
- Borramos la configuración que aparece por defecto con el símbolo de menos (-) hasta que nos quede vacío como la siguiente pantalla:



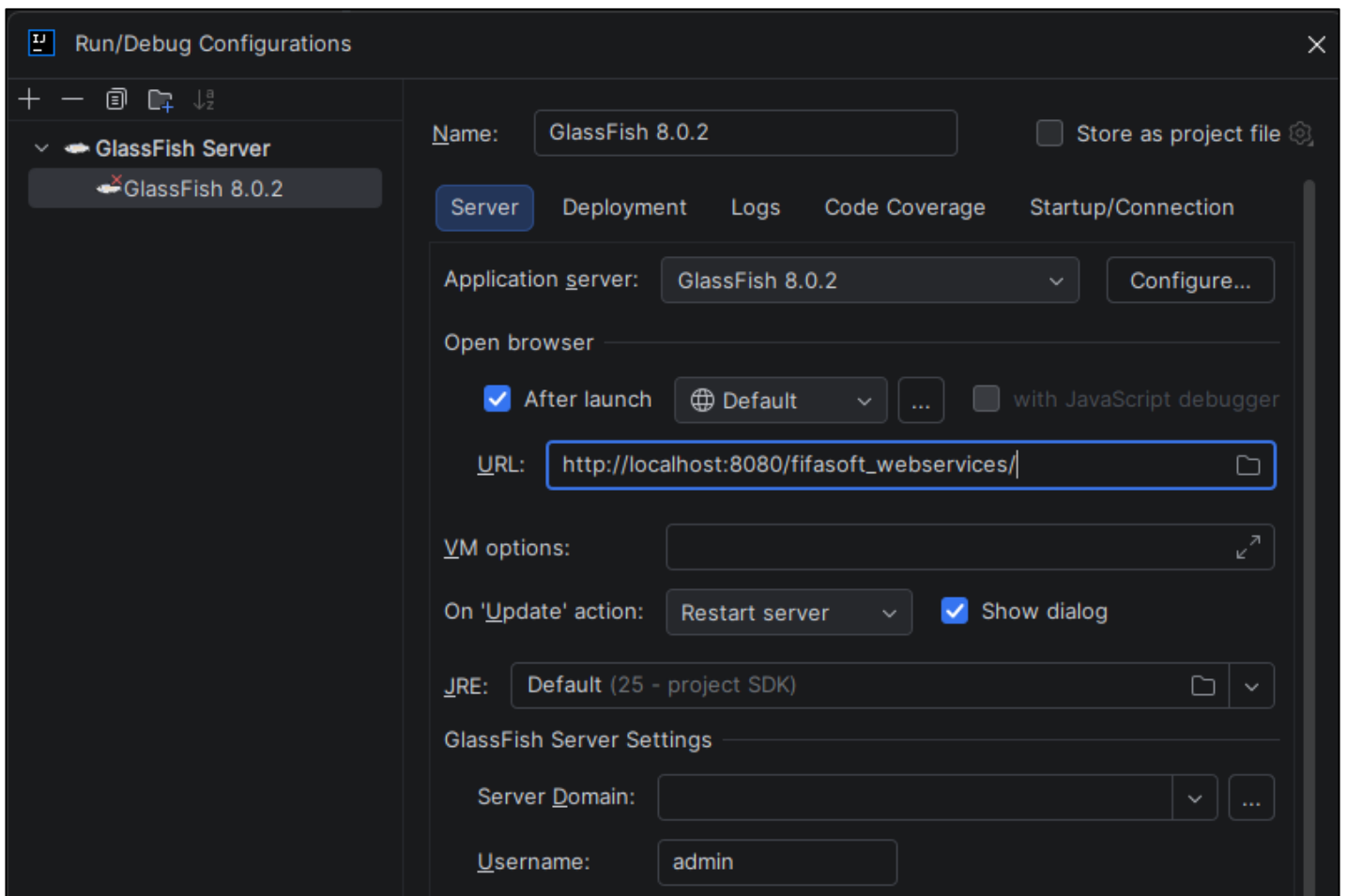
- Ahora le damos clic al botón + y seleccionamos **"GlassFish Server"** -> **"Local"**



- Nos aparecerá lo siguiente:



En la URL colocamos: http://localhost:8080/fifasoft_webservices/



- En la opción de “**Server Domain**” elegimos “**domain1**”.

Run/Debug Configurations

+

–

📄

🔗

↓^a/_z

✓ **GlassFish Server**

✖ GlassFish 8.0.2

Name: GlassFish 8.0.2 ☐ Store as project file

☒ After launch ☐ Default ... ☐ with JavaScript debugger

URL: http://localhost:8080/fifasoft_webservices/

VM options:

On 'Update' action: Restart server ☒ Show dialog

JRE: Default (25 - project SDK)

GlassFish Server Settings

Server Domain: domain1

Username: admin

Password:

☐ Preserve Sessions Across Redeployment

☐ Java EE 5 compatibility

Before launch

+ – 🔗 ⬆️ ⬇️

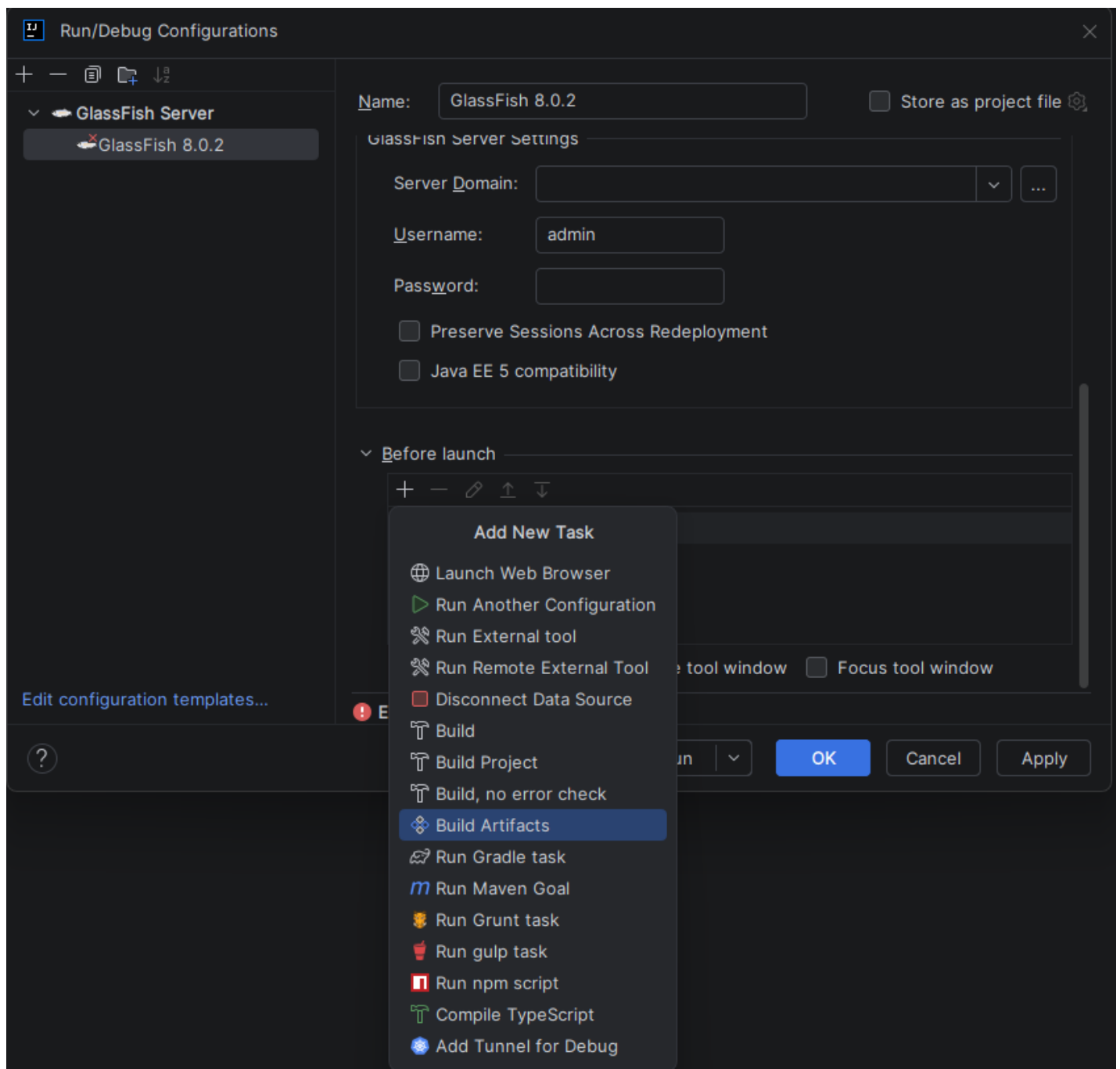
🔧 Build

Edit configuration templates...

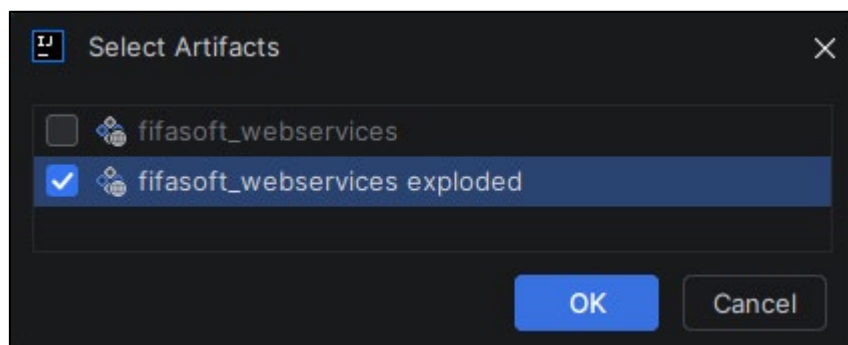
⚠️ Warning: No artifacts marked for deployment

? Run OK Cancel Apply

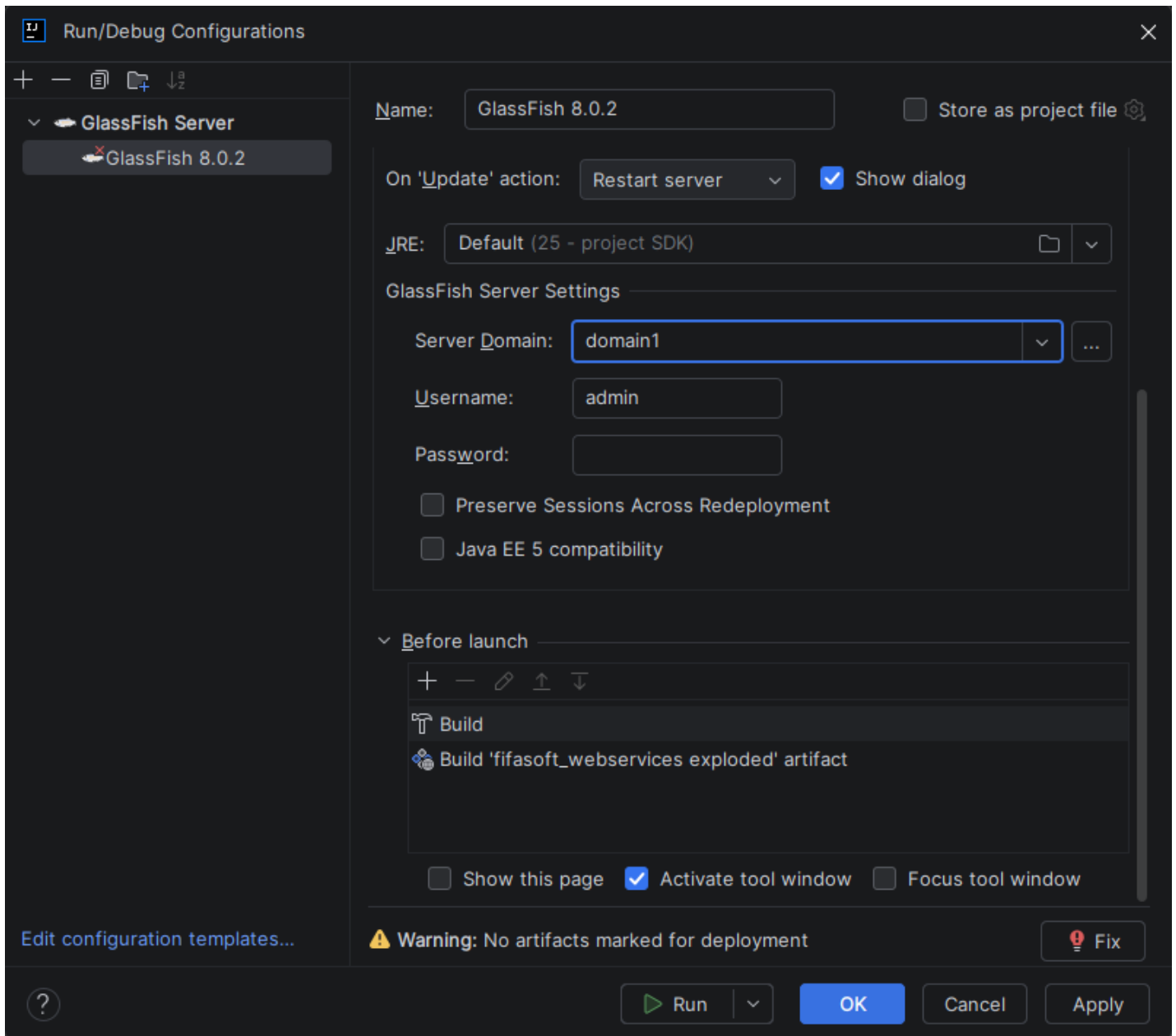
- Luego bajamos y en la parte de “**Before Launch**” le damos clic a + y elegimos “**Build Artifacts**”.



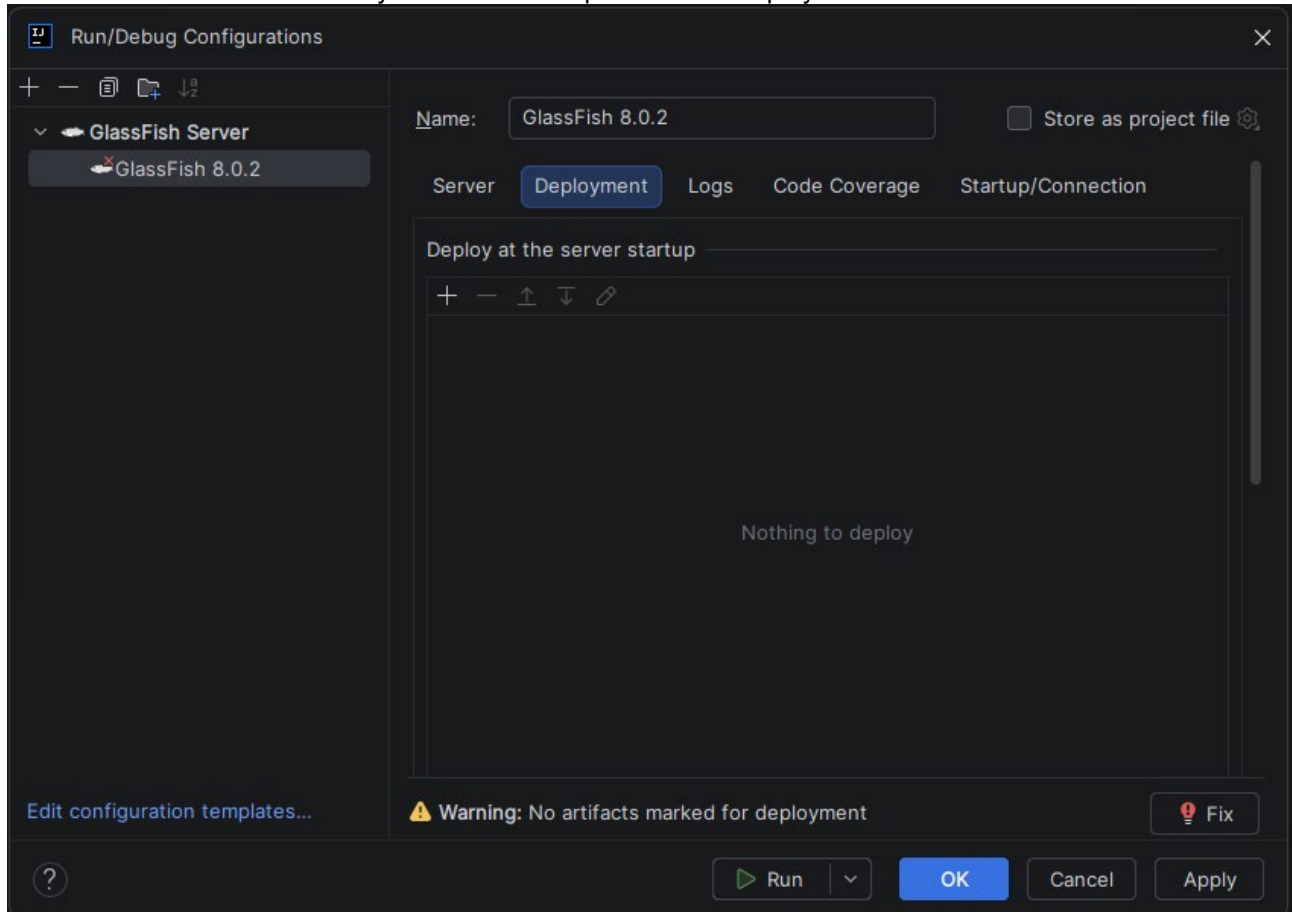
- Elegimos “**fifasoft_webservices exploded**”



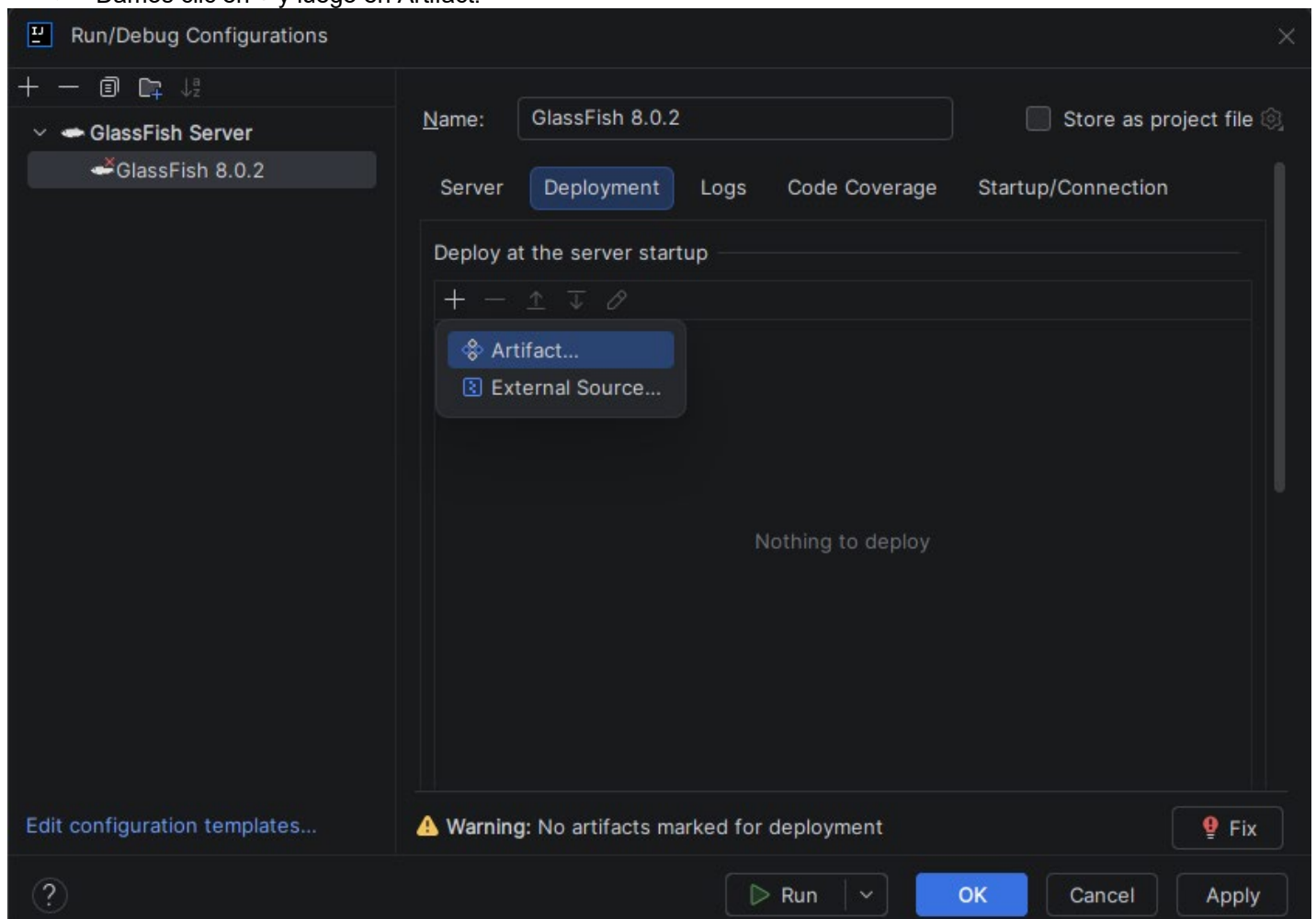
- Quedará de la siguiente manera:



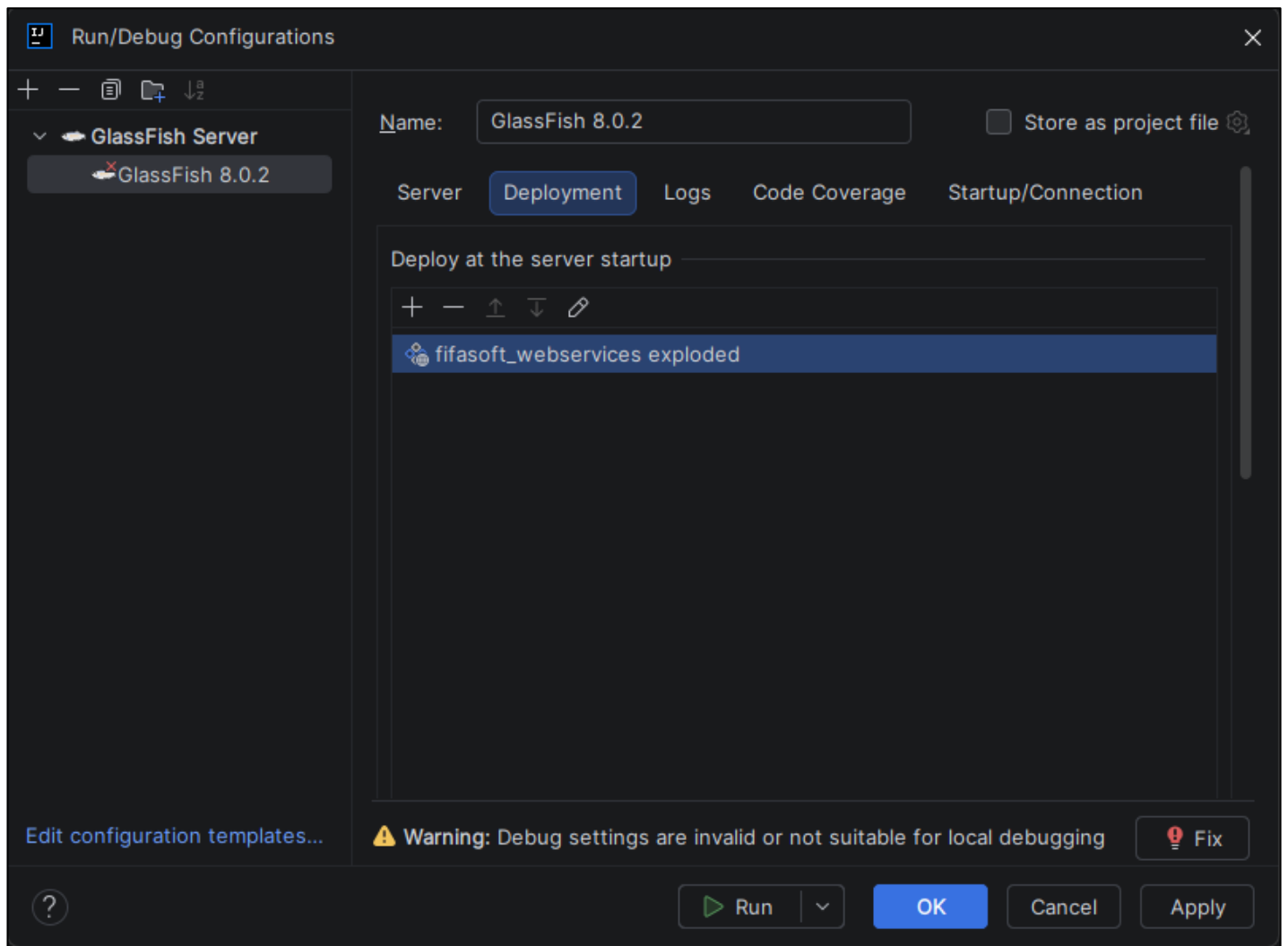
- Ahora subimos nuevamente y nos vamos a la pestaña de “Deployment”.



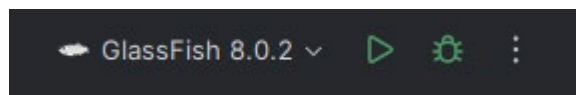
- Damos clic en + y luego en Artifact.



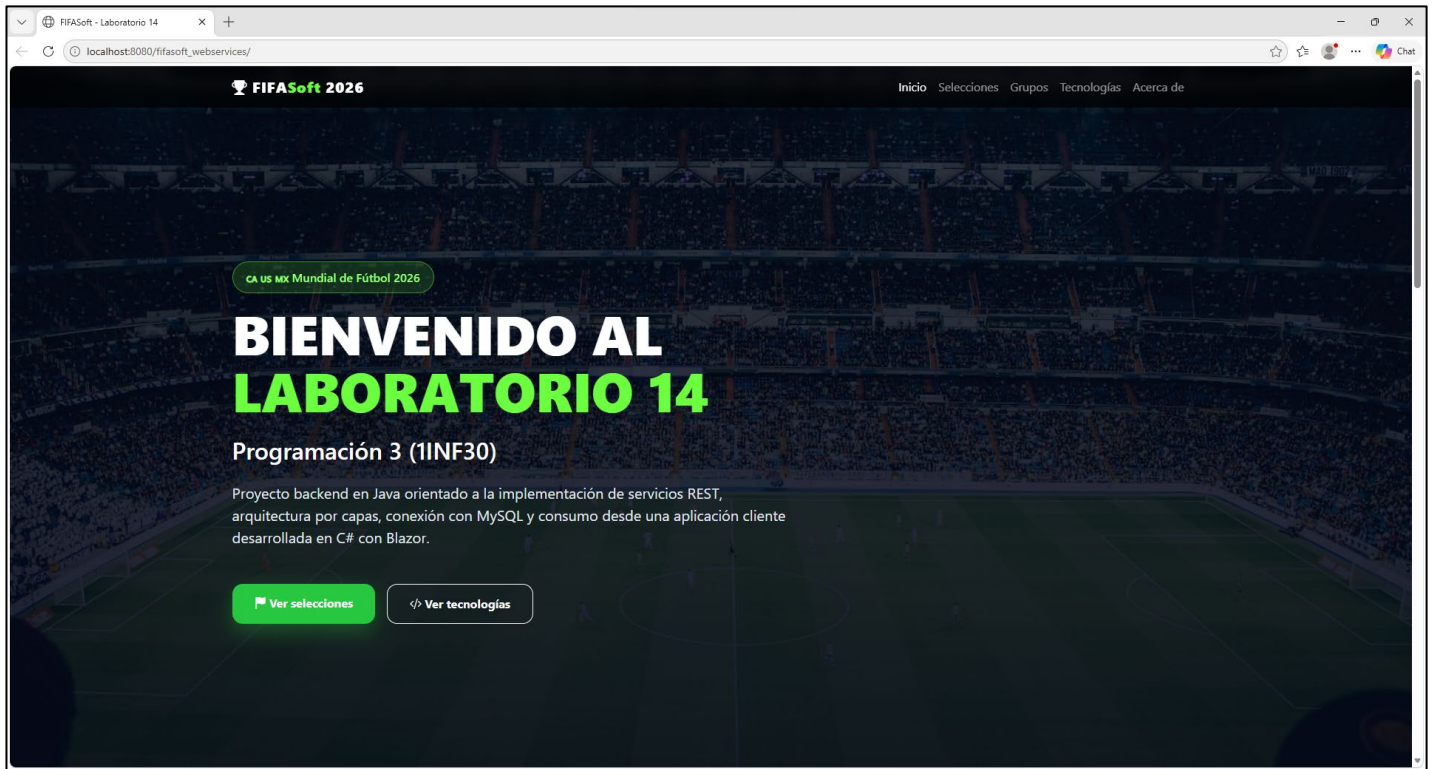
- Agregamos “**fifasoft_webservices exploded**”. Queda de la siguiente manera. Ahora damos clic en “**Apply**” y “**OK**”.



- Ahora damos clic en el botón de **play** en la parte superior:



- Debería cargar la siguiente página web:



- Una vez realizada la configuración, procede con el desarrollo del enunciado.
- El proyecto base JAVA ya incluye todas las referencias y dependencias necesarias para su correcto funcionamiento. En consecuencia, no es necesario crear ni configurar un archivo pom.xml, ni realizar configuraciones adicionales relacionadas con la gestión de dependencias. Deberán concentrarse únicamente en la implementación de las clases y funcionalidades solicitadas en el presente laboratorio.

Caso de estudio:

La FIFA está desarrollando un sistema de información para la visualización de las selecciones clasificadas a la **Copa Mundial de la FIFA 2026**. Para ello, se ha definido el modelo de base de datos mostrado en la Figura 01, el cual servirá como base para el desarrollo del sistema.

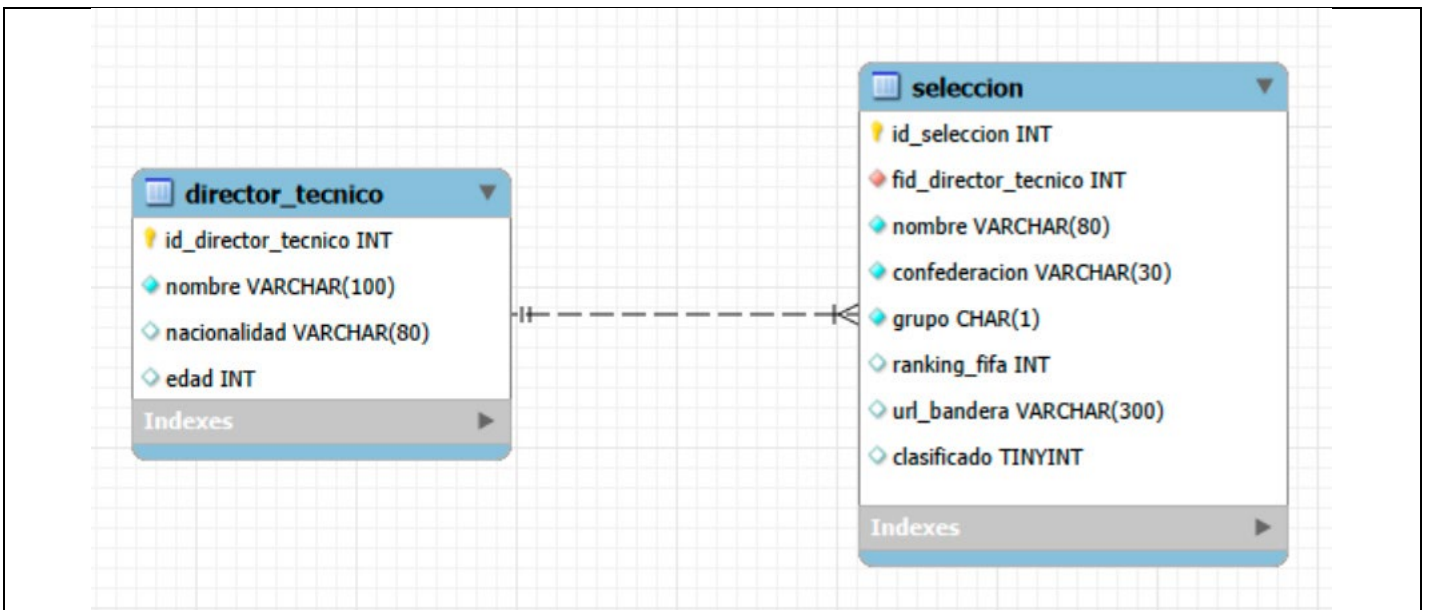


Fig. 01. Diagrama EER del diseño de la base de datos

La solución deberá implementarse utilizando una arquitectura cliente-servidor, con un backend en Java encargado de exponer servicios REST y un frontend en C# Blazor responsable de consumir dichos servicios y mostrar una interfaz similar a la presentada en la Figura 02.

Actualmente, el aplicativo en C# Blazor muestra selecciones cargadas de manera fija en el código, es decir, datos **hardcodeados** que no provienen de la base de datos. Como parte del laboratorio, deberán modificar este comportamiento para que la información mostrada sea obtenida mediante el consumo de los servicios REST desarrollados en el backend Java.

Como parte del material proporcionado, se incluye un script SQL que crea las tablas del sistema y carga los datos iniciales necesarios.

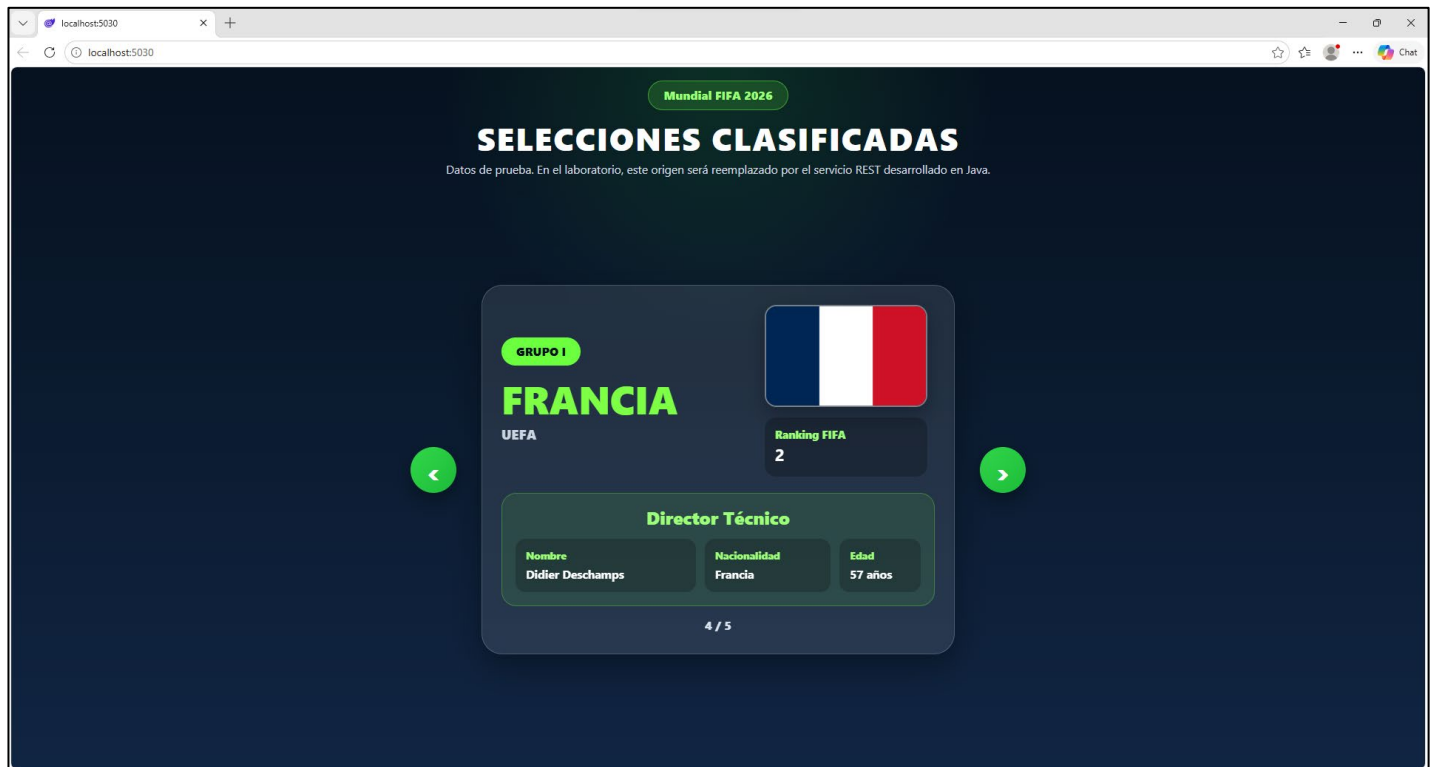


Fig. 02. Página web para visualización de los equipos clasificados

Se solicita completar la implementación de los proyectos base de Java para desarrollar los siguientes servicios REST:

- **Servicio REST DirectorTecnicoRS (8 puntos):** Implementar un servicio que exponga una operación capaz de recibir como parámetro el identificador de un director técnico y devolver toda la información asociada a dicho registro almacenado en la base de datos.
- **Servicio REST SeleccionRS (8 puntos):** Implementar un servicio que exponga una operación para obtener la lista completa de selecciones registradas en la base de datos. Cada selección deberá retornar únicamente los datos propios de la selección, incluyendo el identificador del director técnico asociado.

Se solicita modificar la implementación del proyecto base en C# Blazor para consumir los servicios REST desarrollados en Java y visualizar todas las selecciones registradas en la base de datos. Para ello, el aplicativo deberá consumir inicialmente el servicio SeleccionRS para obtener la lista de selecciones y, posteriormente, por cada selección obtenida, consumir el servicio DirectorTecnicoRS utilizando el identificador del director técnico asociado, con el fin de completar la información mostrada en la interfaz de usuario. **(4 puntos)**

La implementación correspondiente al consumo de los servicios REST en el frontend C# se encuentra centralizada en la clase **SeleccionStateService**, específicamente en el método:

```
public async Task CargarDatosAsync(HttpClient http)
```

Actualmente este método utiliza datos de prueba (hardcodeados) para permitir la visualización inicial de la interfaz. Se solicita reemplazar dicha implementación por el consumo de los servicios REST desarrollados en Java, de manera que la información de las selecciones y de sus respectivos directores técnicos sea obtenida dinámicamente desde el backend.

```

1 referencia
public async Task CargarDatosAsync(HttpClient http)
{
    if (EstaCargado) return;

    // =====
    // DATOS DE PRUEBA
    // =====
    Selecciones = DatosPrueba.ObtenerSelecciones();

    // =====
    // CÓDIGO QUE DEBEN IMPLEMENTAR
    // =====
    /*
    Selecciones =
        .....

    foreach (var seleccion in Selecciones)
    {
        seleccion.DirectorTecnico =
            .....
    }
    */

    EstaCargado = true;
}

```

Tenga en cuenta lo siguiente:

Importante: El archivo **datos.properties** se encuentra incluido por defecto en el proyecto **fifasoft_dbmanager** y **no debe ser movido de la ubicación en la que se entrega**, ya que la instrucción:

```
ResourceBundle db = ResourceBundle.getBundle("datos");
```

realizará su lectura correctamente siempre que el archivo permanezca en dicho lugar.

No obstante, tenga en cuenta que actualmente el archivo se denomina **datos.properties**. Usted puede **modificar el nombre del archivo**, siempre que actualice de manera consistente el parámetro utilizado en `ResourceBundle.getBundle(...)`. Asimismo, puede modificar las **variables** y los **valores** definidos en el archivo con la información correspondiente a su propia conexión a la base de datos, de modo que el **DBManager** (que deberá implementar) funcione correctamente dentro del programa.

Por otro lado, todas las excepciones, mensajes de error e impresiones generadas durante la ejecución del proyecto web **no se visualizarán en la consola de IntelliJ IDEA**. En su lugar, estos registros podrán consultarse en el archivo **server.log**, ubicado en la siguiente ruta:

```
glassfish8\glassfish\domains\domain1\logs\server.log
```

Por tal motivo, se recomienda mantener este archivo abierto con un editor de texto (por ejemplo, **Notepad** o **Notepad++**) mientras se desarrolla y prueba la aplicación. De esta manera, será posible monitorear en tiempo real los mensajes y excepciones emitidos por el servidor de aplicaciones **GlassFish**, facilitando la identificación y solución de errores.

San Miguel, 01 de julio del 2026